

Logistics & Transportation ERP

Complete Implementation Documentation

Project	Logistics & Transportation ERP
Repository	github.com/balaji-001-gif/logistics-and-transportation
App Name	logistics_transport_erp
Framework	Frappe v15 / ERPNext
Date	May 2026
Status	■ Production Ready

1. Executive Summary

The Logistics & Transportation ERP was audited and found to have a solid operational core covering freight settlement, trip sheets, and consignment tracking. However, it lacked the technological integration required for a modern, production-grade logistics platform in India.

This implementation transforms the ERP with government API integrations, intelligent automation, GPS intelligence, and a mobile-first driver portal.

Feature	Status
Centralized API Settings (Logistics Settings)	■ Implemented
Vahan/Sarathi Government API — RC & DL Verification	■ Implemented
Predictive Maintenance Daily Scheduler	■ Implemented
NIC E-Way Bill API Automation	■ Implemented
Google Maps GPS Route Mapping	■ Implemented
Driver Mobile Portal	■ Implemented
Vehicle Profitability Report	■ Implemented
Driver Trip Summary Report	■ Implemented
Automated Notifications (RC Expiry, Maintenance)	■ Implemented
Management Workspace Dashboard	■ Implemented

2. Initial Audit — Before State

Repository Structure (Before)

```
logistics-and-transportation/
├── logistics_transport_erp/
│   ├── hooks.py
│   ├── install.py
│   └── logistics_transportation/
│       └── doctype/ [16 DocTypes]
└── pyproject.toml
```

■ Strengths (Already Present)

- 16 DocTypes covering the full logistics lifecycle
- Freight Order with GST calculations (CGST/SGST/IGST)
- Trip Sheet with Toll & Fuel tracking
- Shipment Tracking Events for real-time status updates
- Vehicle Maintenance Request with parts tracking
- E-Way Bill DocType with expiry alerts
- Driver DL expiry validation on save
- Customer Contract & Rate Card management
- Freight Settlement & Invoice generation
- Load Planning module

■ Gaps Identified

- No government API integration (Vahan, Sarathi, NIC)
- No GPS/GIS capabilities on any form
- No predictive maintenance automation
- No centralized API settings management
- No mobile portal for field drivers
- No management dashboard/workspace
- No analytical reports (profitability, driver KPIs)
- Scheduler registered but tasks.py was empty
- Route Master had no geocoding capability

3. Feature Implementations

Feature 1 — Logistics Settings (Configuration Hub)

A centralized singleton DocType to manage all API credentials and operational thresholds from one place — eliminating hardcoded values across the codebase.

Field	Type	Purpose
nic_api_environment	Select	Sandbox / Production toggle
nic_gstin	Data	Company GSTIN for NIC APIs
nic_password	Password (Encrypted)	AES-256 encrypted NIC credential
vahan_api_key	Password (Encrypted)	Encrypted Vahan/Sarathi API key
vahan_api_enabled	Check	Master toggle for compliance enforcement
google_maps_api_key	Password (Encrypted)	Encrypted Google Maps key
maintenance_alert_days	Int	Days before service → trigger VMR (default: 7)
maintenance_alert_km	Int	KM gap before service → trigger VMR (default: 500)
ewb_expiry_alert_hours	Int	Hours before EWB expiry → alert (default: 6)

Feature 2 — Vahan/Sarathi Government API Integration

Automated real-time verification of vehicle Registration Certificates (RC) and driver Driving Licences (DL) against government databases. Blocks trip dispatch if compliance fails.

New Fields — Vehicle

Field	Type	Description
rc_status	Data (read-only)	Active / Expired / Unknown
rc_fitness_upto	Date (read-only)	Fitness validity from Vahan
rc_owner_name	Data (read-only)	Registered owner from Vahan
rc_verified_on	Datetime (read-only)	Last verification timestamp

New Fields — Driver

Field	Type	Description
dl_verified_status	Data (read-only)	Active / Expired / Unknown
dl_valid_upto	Date (read-only)	DL validity from Sarathi
dl_verified_class	Data (read-only)	Vehicle class from Sarathi
dl_verified_on	Datetime (read-only)	Last verification timestamp

Field	Type	Description
current_vehicle	Link → Vehicle	Auto-set on Trip Sheet submission

Compliance Guard — Trip Sheet Submission

```
def check_compliance_before_dispatch(self):
    settings = frappe.get_single("Logistics Settings")
    if not settings.vahan_api_enabled:
        return # Skip when API disabled (dev/test mode)
    if self.vehicle:
        rc_status = frappe.db.get_value("Vehicle", self.vehicle, "rc_status")
        if rc_status not in ("Active", "API Disabled"):
            frappe.throw("Vehicle RC status invalid – verify via Vahan first.")
    if self.driver:
        dl_status = frappe.db.get_value("Driver", self.driver, "dl_verified_status")
        if dl_status not in ("Active", "API Disabled"):
            frappe.throw("Driver DL status invalid – verify via Sarathi first.")
```

Feature 3 — Predictive Maintenance Scheduler

A daily automated scheduler that proactively monitors vehicle service thresholds and creates maintenance requests before breakdowns occur.

Trigger	Check	Action
Daily midnight	current_odometer >= next_service_km – 500	Create VMR + Alert Fleet Manager
Daily midnight	today >= next_service_date – 7 days	Create VMR + Alert Fleet Manager
Daily midnight	EWB expiring within 6 hours	Send real-time expiry warning

Feature 4 — NIC E-Way Bill API Automation

Direct integration with the GST/NIC E-Way Bill portal to generate EWBs programmatically, eliminating manual portal entry.

Environment	Base URL
Sandbox	https://einvoice1-sandbox.nic.in/EWB/ewbapi
Production	https://einvapi.nic.in/EWB/ewbapi

Token Management: Auth tokens are cached for 5.5 hours (NIC TTL = 6h). On 401 responses, the system auto-refreshes the token and retries the request transparently.

Feature 5 — GPS Route Mapping & Driver Portal

Google Maps Integration

Component	File	Capability
Frontend JS	public/js/route_map.js	Dark-mode route map, tracking event pins
Backend API	api/maps_api.py	Geocoding, Distance Matrix, Reverse Geocode
Route Master	route_master.py	Auto-calculate driving distance & transit days
Tracking Event	shipment_tracking_event.py	Reverse geocode GPS coords to city/state

Driver Portal (/app/driver-portal)

- View all assigned Open & In Transit trips
- One-tap status update (Dispatched, In Transit, Delivered)
- Automatic GPS coordinate capture via browser geolocation
- Log Fuel and Toll expenses that sync directly to the Trip Sheet
- Mobile-first responsive design with dark-mode aesthetics

4. Reports & Analytics

Vehicle Profitability Report

Column	Source	Formula
Revenue	Freight Order.total_amount	SUM (submitted orders)
Fuel Expense	Trip Sheet.total_fuel_amount	SUM (submitted trips)
Toll Expense	Trip Sheet.total_toll_amount	SUM (submitted trips)
Maintenance Cost	VMR.total_maintenance_cost	SUM (completed VMRs)
Net Profit	Calculated	Revenue – (Fuel + Toll + Maintenance)
Margin %	Calculated	Net Profit ÷ Revenue × 100

Driver Trip Summary Report

Column	Source	Formula
Total Trips	Trip Sheet	COUNT (submitted trips)
Total KM	Trip Sheet.distance_covered_km	SUM
Total Fuel (L)	Trip Sheet.total_fuel_litres	SUM
Avg KMPL	Calculated	Total KM ÷ Total Fuel
Advance Taken	Trip Sheet.driver_advance_given	SUM

5. Complete File Inventory

New Files Created (25 files)

api/	
__init__.py	
vahan_api.py	# Vahan/Sarathi API wrapper
ewb_api.py	# NIC E-Way Bill API wrapper
maps_api.py	# Google Maps API wrapper
driver_portal_api.py	# Driver Portal backend API
tasks.py	# All scheduler functions
public/js/route_map.js	# Reusable Google Maps component
logistics_transportation/	
doctype/logistics_settings/	
__init__.py	logistics_settings.json .py .js
notification/	
vehicle_maintenance_due_auto/	vehicle_maintenance_due_auto.json
vehicle_rc_expiry_alert/	vehicle_rc_expiry_alert.json
report/	
vehicle_profitability/	.json .py .js
driver_trip_summary/	.json .py
page/driver_portal/	
driver_portal.json	.html .js .css
workspace/logistics_management/	
logistics_management.json	

Modified Files (12 files)

File	Changes Made
hooks.py	+ scheduler events, + Workspace fixture, + route rules
vehicle.json	+ rc_status, rc_fitness_upto, rc_owner_name, rc_verified_on, maintenance_alert_sent_on
vehicle.py	+ verify_rc() method + verify_vehicle_rc() whitelist endpoint
vehicle.js	+ Verify RC button + colour status indicators
driver.json	+ dl_verified_status, dl_valid_upto, dl_verified_class, dl_verified_on, current_vehicle
driver.py	+ verify_dl() method + verify_driver_dl() whitelist endpoint
driver.js	+ Verify DL button + colour status indicators

File	Changes Made
trip_sheet.json	+ route_map_html HTML field
trip_sheet.py	+ check_compliance_before_dispatch() + update_driver_status()
trip_sheet.js	+ GPS map rendering from load plan route
freight_order.json	+ route_map_html HTML field
freight_order.js	+ GPS map rendering + live tracking pins
e_way_bill.py	+ generate_ewb_via_api() + generate_e_way_bill() whitelist
e_way_bill.js	+ Generate E-Way Bill (NIC API) button
route_master.json	+ lat_origin, lng_origin, lat_destination, lng_destination, route_map_html
route_master.py	+ fetch_coordinates_from_api() + fetch_route_coordinates() whitelist
route_master.js	+ Fetch Coordinates & Distance button + map render
shipment_tracking_event.py	+ reverse_geocode_if_needed() on validate

6. Deployment Guide

Step 1 — Clean Server Installation

```
cd ~/f15-bk
# Remove any previous failed attempt
rm -rf apps/logistics_transport_erp
rm -rf apps/logistics-and-transportation

# Pull app from GitHub
bench get-app https://github.com/balaji-001-gif/logistics-and-transportation.git
```

Step 2 — Install & Migrate

```
bench --site [your-site-name] install-app logistics_transport_erp
bench --site [your-site-name] migrate
bench build --app logistics_transport_erp
bench restart
```

Step 3 — Configure API Settings

Navigate to **Logistics Management** → **Logistics Settings** and configure:

Section	Setting	Value
NIC E-Way Bill	Environment	Sandbox (testing) / Production
NIC E-Way Bill	GSTIN	Your 15-digit company GSTIN
NIC E-Way Bill	Username / Password	NIC portal credentials
Vahan/Sarathi	API Key	Key from NIC developer portal
Vahan/Sarathi	Enable	☒ Check to enforce compliance
Google Maps	API Key	Google Cloud Console API Key
Google Maps	Enable	☒ Check to show maps on forms
Alert Thresholds	Maintenance Alert Days	7
Alert Thresholds	Maintenance Alert KM	500
Alert Thresholds	EWB Expiry Hours	6

Step 4 — Verify

```
# Test maintenance scheduler manually
bench --site [site] execute logistics_transport_erp.tasks.auto_maintenance_check
```

```
# Check for errors
```

```
tail -f logs/frappe.log
```

7. Before vs After Comparison

Capability	Before	After
Government API Verification	■ Manual/None	■ Vahan RC + Sarathi DL (1-click)
Compliance Enforcement	■ None	■ Blocks trip dispatch if invalid
Predictive Maintenance	■ Manual only	■ Automated daily scheduler
E-Way Bill Generation	■ Manual portal entry	■ Direct NIC API from ERP form
Route Visualization	■ None	■ Google Maps on all trip forms
Distance Calculation	■ Manual/estimated	■ Auto via Google Distance Matrix
Real-time GPS Tracking	■ Text descriptions only	■ Live pins plotted on map
Driver Field Access	■ None	■ Mobile-first Driver Portal
Vehicle Profitability	■ No reports	■ Automated Script Report
Driver Performance KPIs	■ No reports	■ KPI Summary Report
Maintenance Alerts	■ Manual	■ Auto email + system notification
RC Expiry Alerts	■ None	■ Automated email to Fleet Manager
Management Dashboard	■ None	■ Dedicated Workspace with shortcuts